

Cambios realizados respecto a la rama main:

MODIFICACIONES

1. Tamaño de memoria compartida dinámico

Versión original:

```
shmget(1234, 298220, IPC_CREAT | 0666);
```

Versión corregida:

```
int tam = PARKING_getTamaNoMemoriaCompartida();  
shmget(IPC_PRIVATE, tam, IPC_CREAT | 0600);
```

El valor 298220, la biblioteca puede cambiar ese tamaño entre versiones o configuraciones. Si el tamaño reservado es menor del necesario, la biblioteca escribirá fuera de la memoria asignada, corrompiendo datos o causando un fallo de segmentación (segfault). La función `PARKING_getTamaNoMemoriaCompartida()` existe precisamente para que el programa siempre pida el tamaño correcto.

2. Número de semáforos dinámico

Versión original:

```
semget(5678, 10, IPC_CREAT | 0666);
```

Versión corregida:

```
int nSem = PARKING_getNSemAforos();  
semget(IPC_PRIVATE, nSem, IPC_CREAT | 0600);
```

Un poco como antes, si se crean menos semáforos de los que la biblioteca necesita (en este caso teníamos 10), cualquier operación `semop` sobre un índice inexistente fallará con error. La función `PARKING_getNSemAforos()` devuelve el número exacto requerido, así evitamos asumir un valor que puede ser incorrecto.

3. Claves IPC fijas a IPC_PRIVATE

Versión original:

```
shmget(1234, ...)  
semget(5678, ...)
```

Versión corregida:

```
shmget(IPC_PRIVATE, ...)  
semget(IPC_PRIVATE, ...)
```

Las claves numéricas fijas como 1234 y 5678 son globales al sistema. Si otro proceso, o una ejecución anterior que no limpió bien sus recursos, ya creó recursos con esas claves, shmget y semget devolverán el ID de ese recurso, con tamaño y permisos incorrectos. Con IPC_PRIVATE siempre creamos un recurso nuevo y exclusivo para el proceso actual.

4. Permisos 0666 a 0600

Práctica más correcta y segura, solo el propio usuario puede leer y escribir. De todas maneras no se hasta que punto aquí puede marcar algún tipo de diferencia.

5. Buzón de mensajes creado correctamente

Versión original:

```
PARKING_inicio(..., 0, ...)
```

Versión corregida:

```
int id_buzon = msgget(IPC_PRIVATE, IPC_CREAT | 0600);  
PARKING_inicio(..., id_buzon, ...)
```

PARKING_inicio recibe el ID del buzón como parámetro y lo usa internamente para la comunicación entre procesos (entre el gestor y los chóferes). 0 no es un ID válido, ya que podría corresponder a un recurso del sistema o simplemente no existir, acabando en fallos. Es necesario crear el buzón con msgget y pasarle su ID real.

6. Función de llegada: int f(int) a int f(HCoche)

Versión original:

```
int mi_llegada_prueba(int id) { ... }
```

Versión corregida:

```
int mi_llegada_prueba(HCoche hc) { ... }
```

TIPO_FUNCION_LLEGADA está definido en parking.h como un puntero a función que recibe HCoche hc.

7. Limpieza del buzón añadida

```
msgctl(id_buzon, IPC_RMID, NULL);
```

AL no tener buzón antes, tampoco limpiábamos correctamente todos los recursos.

AÑADIDOS

1. Función para gestión de argumentos

La función validar_argumentos valida que, se introduzcan mínimo 3 argumentos y máximo 5 (./programa ret chof como mínimo y ./programa ret chof D PA (o PD en su defecto) como máximo).

Después, a partir del argumento chof, identifica si la cadena introducida corresponde a D, PA o PD para comprobar si se introdujeron los argumentos correctos. Finalmente comprueba si PA y PD se introdujeron juntos, lo cual no es válido.

2. Variables globales

```
int retardo, num_choferes, debug, prio_PA, prio_PD;  
int id_mem = -1, id_sem = -1, id_buzon = -1;
```

Para que sea más sencillo gestionar luego el paso de parámetros a los diferentes procesos. Lo suyo sería crear un struct con todos ellos pero de momento lo dejo así, al final no es difícil cambiarlo después.

Las tres del final las inicio en -1 por que así se puede saber que no se crearon todavía esos recursos.